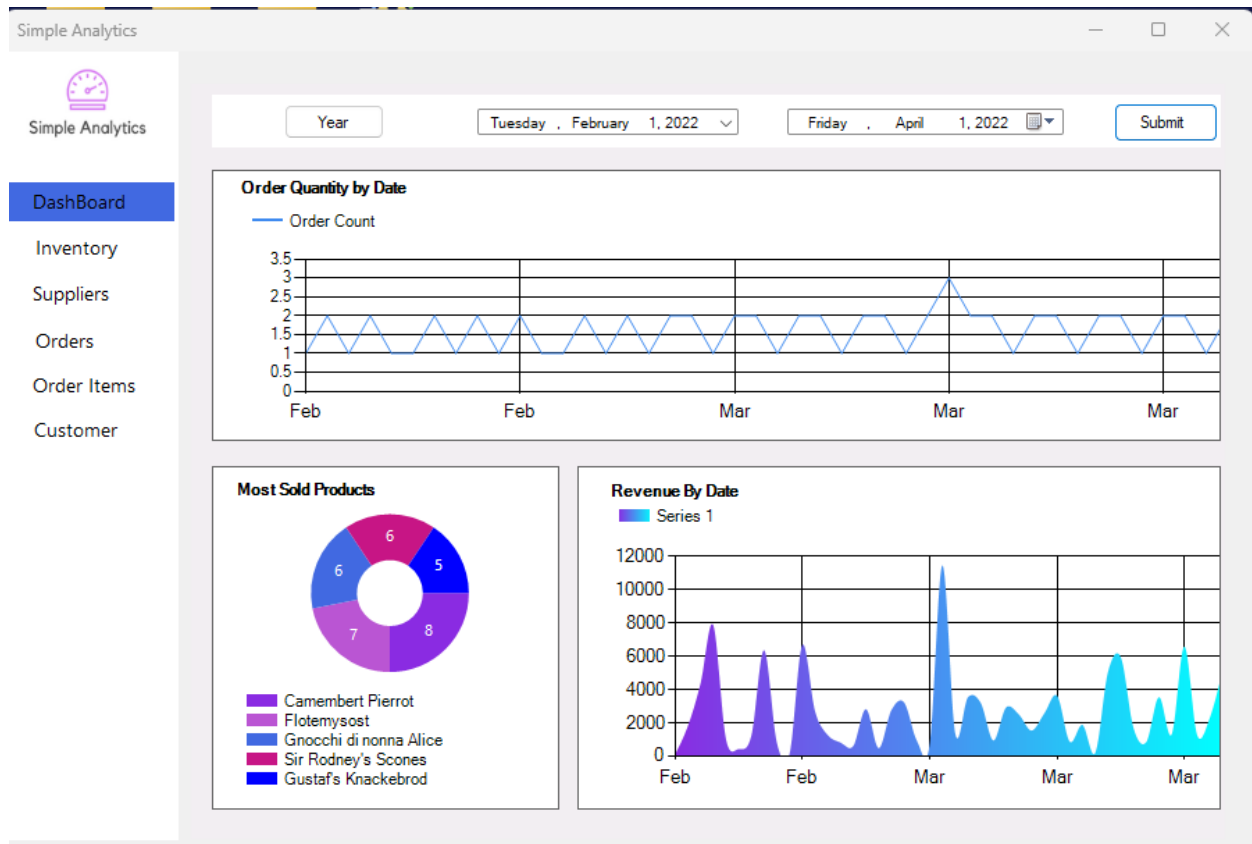


Program Architecture

The inventory management program is Winforms restaurant inventory management system that's split into 2 applications. It separates the data layer from the UI layer.



Finance Manager- FinanceManager is the UI layer and the main application. It contains all WinForms screens, UI logic, CRUD operations, repositories, and the application entry point (**Program.cs**). It launches the program, handles user interaction, and communicates with the data layer through model objects and repository classes.

InventoryApplication.Models - contains models, interfaces and shared logic the Winforms app uses. This is the data layer.

Finance manager references **InventoryApplication.Models** which is how it talks to that project.

Finance Manager-

First, `program.cs` lives in this program, which starts the whole application, loads the first form and controls the whole lifetime of the app.

The first form that loads in the application is **Form1**, which acts as the main dashboard and the central controller for the entire UI. This form is responsible for:

- Loading all data when the application starts
- Displaying inventory, suppliers, orders, order items, and customers
- Handling all CRUD operations through buttons and dialogs
- Refreshing `DataGridViews` after changes
- Running searches
- Switching between tabs
- Loading dashboard analytics (charts)
- Subscribing to events from Add/Edit forms

How Form1 Loads Dashboard Analytics

Form1 uses a separate class called **DashboardRepository** to load data for the charts on the dashboard. This repository does not talk directly to SQL - instead, it uses the existing repositories (`OrderRepository`, `ProductRepository`, `OrderItemRepository`) to pull data and then performs LINQ operations to group, filter, and aggregate the results.

This keeps the analytics logic clean and separate from the UI.

DashboardRepository

The `DashboardRepository` is how I generate the data that appears in the dashboard graphs. It uses LINQ to group and aggregate data that comes from the repositories.

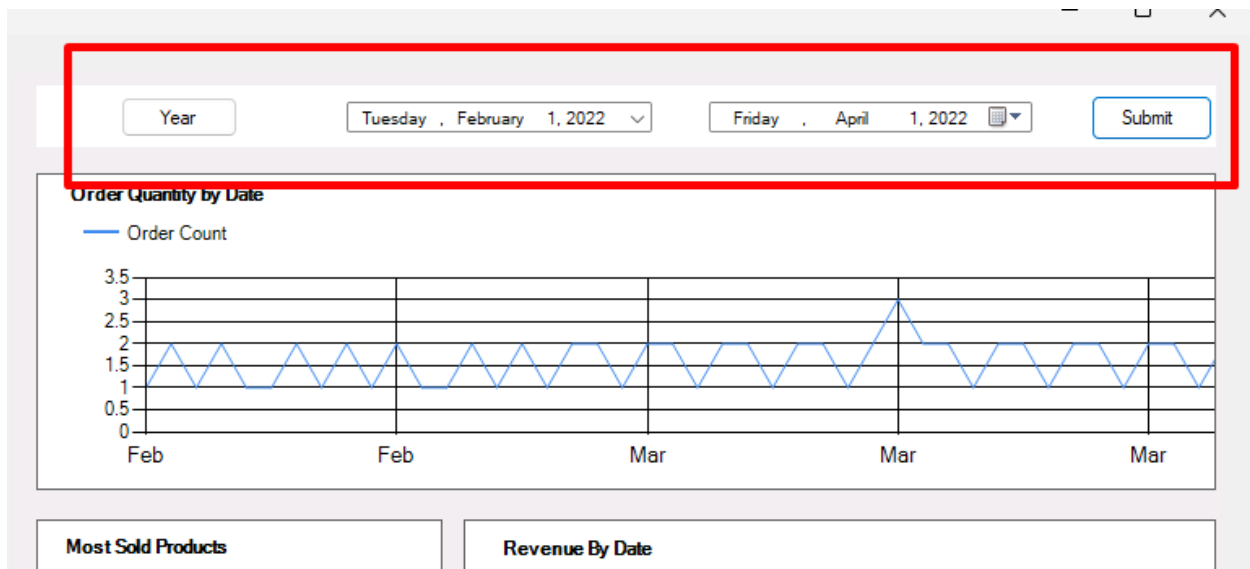
✓ GetOrdersByDate

This method returns a list of `OrdersByDate` objects. I filter orders by date range, group them by date, count them, and return the results in a format the chart can bind to.

```
//returns key value pair for graph
1 reference
public static List<OrdersByDate> GetOrdersByDate(DateTime startDate, DateTime endDate)
{
    //returning a list of orders where date condition is met.
    // p representing object we get from our list (object that is returned). it represents an order from the returned list of orders
    // getting a collection and pushing it into gross orders list.
    var grossOrdersList = OrderRepository.Instance.GetAll()
        .Where(p => p.OrderDate >= startDate && p.OrderDate <= endDate);

    List<OrdersByDate> groupedOrders = grossOrdersList.GroupBy(p => p.OrderDate).OrderBy(p
        => p.Key).ToDictionary(p => p.Key, p => p.Count())
        .Select(p => new OrdersByDate { Date = p.Key, TotalAmount = p.Value }).ToList();

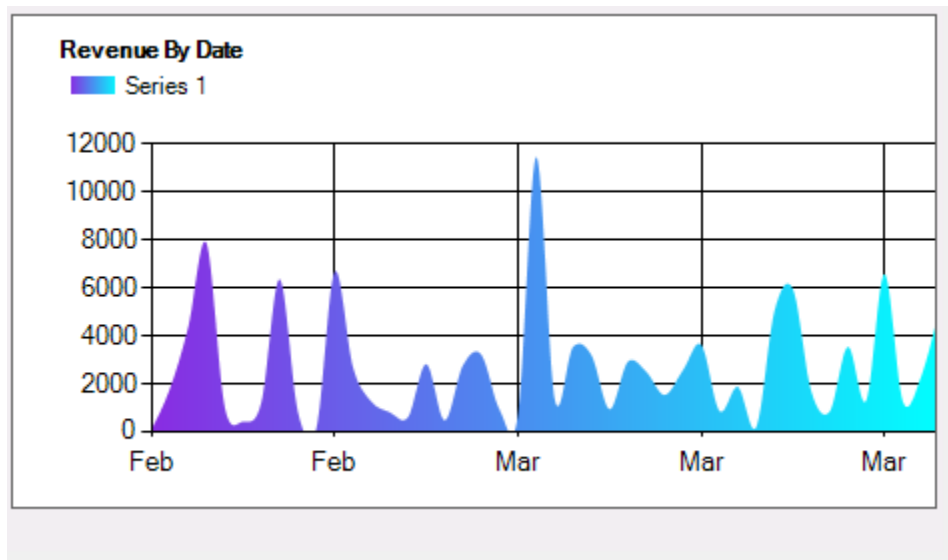
    return groupedOrders;
}
```



✓ RevenueByDate

This method calculates total revenue per day. I group orders by date and sum the TotalAmount field.

```
1 reference
public static List<RevenueByDate> RevenueByDate(DateTime startDate, DateTime endDate)
{
    var grossOrdersList = OrderRepository.Instance.GetAll()
        .Where(p => p.OrderDate >= startDate && p.OrderDate <= endDate);
    List<RevenueByDate> groupedOrders = grossOrdersList.GroupBy(p => p.OrderDate).OrderBy(p =>
        p.Key).ToDictionary(p => p.Key, p => p.Sum(s => s.TotalAmount))
        .Select(p => new RevenueByDate { Date = p.Key, TotalAmount = p.Value }).ToList();
    return groupedOrders;
}
```



TopProducts

This method returns the top 5 products sold in the selected date range. It uses:

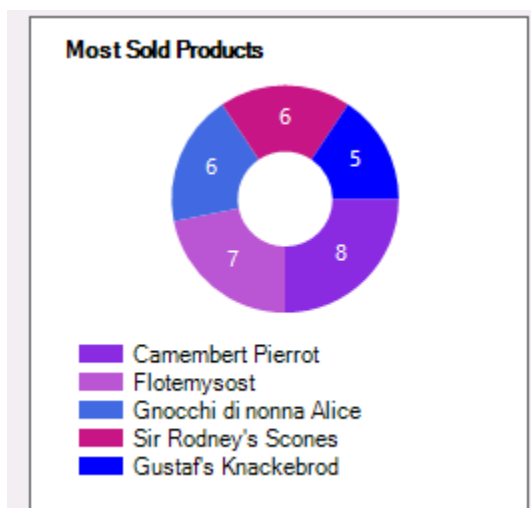
- Order items
- Orders
- Products
- LINQ joins
- Grouping
- Sorting
- Projection into a TopProducts model

This is how I generate the “Top 5 Products” chart.

```
1 reference
public static List<TopProducts> TopProducts(DateTime startDate, DateTime endDate)
{
    var grossOrdersItems = OrderItemRepository.Instance.GetAll();
    var products = ProductRepository.Instance.GetAll();
    var orders = OrderRepository.Instance.GetAll();

    List<TopProducts> groupedOrders = grossOrdersItems

    //doing an inner join on orderitems.id and orders.id
    .Join(orders, o => o.OrderId, p => p.Id,
        // casting o and p into one item and make a new object. haven't defined this item. anonymous item.
        //so the object has o and p as properties
        (o, p) => new { o, p })
    // doing where clause on new object- it's a list of everything that we get back from the inner join above. p is o.p.
    // 2nd p represents orders and thats where we are going to get the oprser date. so out of this inner join only g ive us what meets condition.
    .Where(p => p.p.OrderDate >= startDate && p.p.OrderDate <= endDate)
    // group all of the records by product id next like this object will be represented by s.
    .GroupBy(p => p.o.ProductId)
    // do another join on products and product id. "first" allows us to get the first row to get a property in the new object so we can get produ
    .Join(products, s => s.First().o.ProductId,
        //inner joining on order.product id and product.id and creating a new object again. (s from order items and p for product)
        p => p.Id, (s, p) => new { s, p })
    // with new object we creating, doing a select columns
    // lambda expression casts out k top product type with corresponding properties.
    .Select(k => new TopProducts
    { ProductID = k.s.Key, ProductName = k.p.ProductName, TotalCount = k.s.Count() })
    .OrderByDescending(p => p.TotalCount)
    .Take(5)
    .ToList();
    return groupedOrders;
}
```



How Form1 Uses DashboardRepository

When the app loads, Form1 calls:

- DashboardRepository.GetOrdersByDate(...)
- DashboardRepository.RevenueByDate(...)
- DashboardRepository.TopProducts(...)

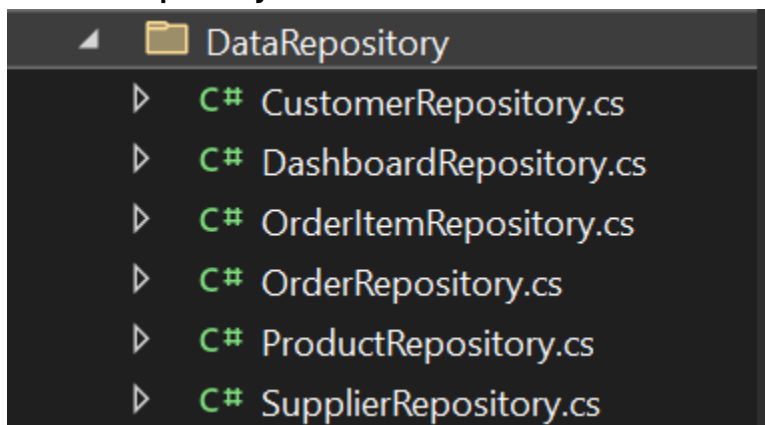
Then it binds the results to:

- chart_number_orders
- chartRevenue
- chart_Top_5

This is how the dashboard analytics appear immediately when the program starts.

Data Repository-

The DataRepository folder contains classes that talk to the database.



DataRepository

Classes that:

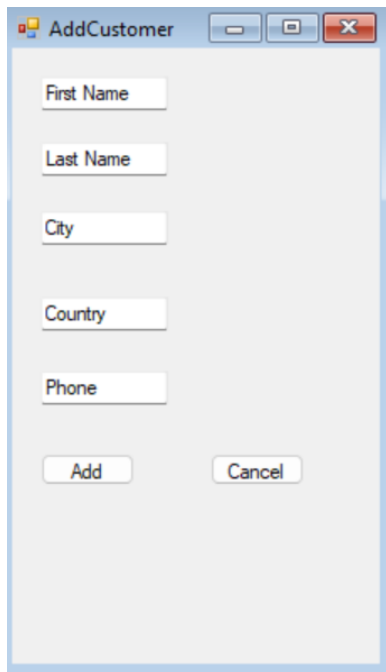
- Receive model objects
- Use **ADO.NET** (SqlConnection, SqlCommand)
- Run **raw SQL**
- Perform CRUD
- Return results to the forms

Add Customer example

1. User fills textboxes
2. Form creates a Customer object
3. Form calls `CustomerRepository.Instance.Add(customer)`
4. Repository uses **ADO.NET** to run an INSERT
5. Customer is saved to the database

Add customer example:

When the user interacts with the form to add a customer and clicks “add”,

A screenshot of a Windows application window titled "AddCustomer". The window has a standard Windows title bar with minimize, maximize, and close buttons. Inside the window, there are five text input fields stacked vertically, labeled "First Name", "Last Name", "City", "Country", and "Phone". At the bottom of the form, there are two buttons: "Add" and "Cancel".

the form calls the repository through the `btnAdd_Click` logic.

Some of the error handling:

```
private void btnAdd_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtFirstName.Text))
    {
        MessageBox.Show("First name can not be empty.");
        return;
    }

    if (string.IsNullOrEmpty(txtLastName.Text))
    {
        MessageBox.Show("Last Name can not be empty.");
        return;
    }

    if (string.IsNullOrEmpty(txtCity.Text))
    {
        MessageBox.Show("City can not be empty.");
        return;
    }
}
```

Code creates a customer object in memory and fills it with the values from the textboxes on the form. From here we have one customer object with the info provided.

The code then calls the customer repository, passing in the customer object. “Add(customer)”

```
Customer customer = new Customer()
{
    FirstName = txtFirstName.Text,
    LastName = txtLastName.Text,
    City = txtCity.Text,
    Country = txtCountry.Text,
    Phone = txtPhone.Text
};

CustomerRepository.Instance.Add(customer);
MessageBox.Show($"Customer: {customer.FirstName + customer.LastName} added");
onCustomerAdded?.Invoke();
this.Close();
this.Dispose();
```

Now the repository has the data the user passed in to add a customer to the database manually with SQL using raw SQL + SqlConnection + SqlCommand. (ADO.NET)

```

public class CustomerRepository : ICustomerRepository
{
    private static CustomerRepository instance;
    5 references
    public static CustomerRepository Instance
    {
        get
        {
            if (instance == null)
            {
                instance = new CustomerRepository();
            }
            return instance;
        }
    }

    1 reference
    public CustomerRepository()
    {
        _connectionString = ConfigurationManager.ConnectionStrings["SqlConnection"].ToString();
    }

    private readonly string _connectionString;
    2 references
    public void Add(Customer customer)
    {
        using (var connection = new SqlConnection(_connectionString))
        using (var command = new SqlCommand("Insert into Customer (FirstName, LastName, City, Country, Phone) " +
            "values (@FirstName, @LastName, @City, @Country, @Phone)", connection))
        {
            command.Parameters.Add(new SqlParameter("@FirstName", customer.FirstName));
            command.Parameters.Add(new SqlParameter("@LastName", customer.LastName));
            command.Parameters.Add(new SqlParameter("@City", customer.City));
            command.Parameters.Add(new SqlParameter("@Country", customer.Country));
            command.Parameters.Add(new SqlParameter("@Phone", customer.Phone));
            connection.Open();
            command.ExecuteNonQuery();
        }
    }
}

```

The DbConnection class stores the connection string, creates a SqlConnection and lets other repository classes inherit so they don't repeat the same code.


```

using System;
using System.Collections.Generic;
using System.Data.SqlClient;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace FinanceManager.Db
{
    1 reference
    public abstract class DbConnection
    {
        private readonly string connectionString;

        0 references
        public DbConnection()
        {
            connectionString = "Server=(local); DataBase=BusinessData; Integrated Security=true";
        }

        0 references
        public SqlConnection GetConnection()
        {
            return new SqlConnection(connectionString);
        }
    }
}

```

InventoryApplication.Models - job is to define the shape of the applications data and the contracts the repositories must follow.

Models- The C# classes like:

- Customer
- Supplier
- Inventory
- Order

That represent the data structure the program uses. They are just blueprints.

Interfaces- Define the contracts for the repositories, which tells finance manager that the customerRepository HAS to implement these methods, keeping architecture clean and consistent.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using InventoryApplication.Models;
//ICustomerRepository is a parent, and CustomerRepository is a child of this parent class.
namespace InventoryApplication.Models.Interfaces
{
    1 reference
    public interface ICustomerRepository
    {
        2 references
        void Add(Customer customer);
        2 references
        void Update(Customer customer);
        2 references
        void Delete(int id);

        2 references
        List<Customer> GetAll();
        2 references
        List<Customer> GetAll(string firstName);
        1 reference
        Customer Get(int customerId);
    }
}
```